



UNIVERSIDAD TECNOLÓGICA NACIONAL
Facultad Regional Tucumán

Patrones GRASP

Asignatura: Diseño de Sistemas

Docentes

Ing. Bustos Thames, Juan

Ing. Burgos, Carla

Integrantes

Albarracín, Ulises

Cáceres, Xavier

Díaz, Dylan

Díaz, Johana

Vilariño, Bruno

Comisión: 3K02

Año: 2025

Índice

Introducción a Patrones GRASP.....	4
Patrón Controlador.....	5
El problema.....	5
La solución.....	5
Tipos o Variantes.....	6
Ejemplos.....	6
Pros y Contras.....	7
 Patrón de Polimorfismo.....	 8
El problema.....	8
La solución.....	8
Tipos o Variantes.....	9
Ejemplos.....	9
Ejemplo cotidiano.....	9
Diagrama UML.....	9
Ejemplo con código.....	10
Pros y Contras.....	11
Beneficios:.....	11
Desventajas:.....	11
 Patrón de Fabricación Pura.....	 12
El problema.....	12
La solución.....	12
Analogía: Crear la entidad CreadorDeJuegos.....	13
Tipos o Variantes.....	13
Ejemplos.....	13
Pros y Contras.....	15
Beneficios.....	15
Desventajas.....	15
 Patrón de Indirección.....	 16
El problema.....	16
En el Software: Alto Acoplamiento.....	16
Analogía: Comprar en el Extranjero.....	16
La solución.....	17
Definición Técnica: El Intermediario.....	17
Analogía: Usando una Tarjeta de Crédito.....	17
Tipos o Variantes.....	18
Fachada.....	18
Adaptador (Adapter).....	18

Ejemplos.....	18
Ejemplo Cotidiano: El Enchufe Eléctrico.....	18
Ejemplo de Código: Terminal de Venta.....	18
Diagrama UML de la Solución.....	19
Pros y Contras.....	19
Beneficios.....	19
Desventajas (Contras).....	20
Patrón de Variaciones Protegidas.....	21
El problema.....	21
La solución.....	21
Tipos o Variantes.....	22
Ejemplos.....	22
Ejemplo cotidiano.....	22
Ejemplo en software (nuevo: sistema de pagos).....	23
Pros y Contras.....	24
Conclusión.....	25
Fuentes.....	26

Introducción a Patrones GRASP

En el diseño de sistemas orientados a objetos, una de las tareas más importantes es la **correcta asignación de responsabilidades a las clases y objetos**. Si estas decisiones no se toman con criterio, el resultado suele ser un software frágil, difícil de mantener y de extender.

Para evitarlo, se utilizan los **patrones GRASP (General Responsibility Assignment Software Patterns)**, los cuales constituyen un conjunto de principios que guían la distribución de responsabilidades de manera sistemática y racional.

Estos patrones no aportan ideas nuevas, sino que **formalizan y nombran prácticas probadas en el diseño de objetos**, facilitando así su comprensión, enseñanza y comunicación. Su aplicación se da principalmente al construir diagramas de interacción, donde se define cómo colaboran los objetos para cumplir con los requerimientos del sistema.

En este documento, se expondrán 5 de los principales patrones GRASP: **Controlador, Polimorfismo, Fabricación Pura, Indirección y Variaciones Protegidas**. Con el fin de comprenderlos para su utilización en nuestro futuro como diseñadores de sistemas.

Patrón Controlador

El problema

Problema en software: ¿Quién debe recibir y coordinar los eventos o peticiones del sistema (usualmente desde la interfaz de usuario)? Si dejamos que la UI haga todo, se rompe la separación de responsabilidades.

Analogía con la vida real:

Imaginemos un **recepcionista en un hospital**. Cuando llega un paciente, la recepcionista no lo atiende directamente ni hace la cirugía; en cambio, recibe la solicitud, la registra y deriva la tarea al área correspondiente (consultorio, laboratorio, etc.).

La solución

- **Definición (Larman):**

Asignar la responsabilidad de manejar un evento del sistema a una clase Controlador, que representa:

- El **sistema** en sí mismo, o
- Un **caso de uso particular**.

- **Explicación técnica:**

- El controlador actúa como intermediario entre la **interfaz de usuario** y la **lógica de negocio**. No debe contener la lógica de negocio, sino delegarla a otras clases responsables (siguiendo Experto en Información, Baja Cohesión, etc.).
- Centraliza la recepción de mensajes, asegurando que la UI permanezca simple y que la lógica esté en el dominio.

- **Analogía con la vida real (hospital):**

El recepcionista (Controlador) recibe al paciente (evento), lo registra y deriva al especialista correcto (clases de dominio). No trata de hacer todo él mismo.

Tipos o Variantes

1. Controlador de casos de uso (Use-Case Controller):

- Una clase para cada caso de uso.
- Ejemplo: `RegistrarVentaController`, `PagarReservaController`.
- Se usa cuando se quiere una separación clara entre diferentes procesos de negocio.

2. Controlador de fachada (Facade Controller):

- Una clase única que representa al sistema en general.
- Ejemplo: `SistemaVentas`.
- Útil cuando el sistema es pequeño o se quiere simplicidad en el modelo.

Ejemplos

En un restaurante, el **mozo** (controlador) recibe los pedidos de los clientes (eventos) y luego los comunica a la cocina (clases de negocio). El mozo no cocina, solo coordina.

Ejemplo en código (Java):

```
// Interfaz de usuario
public class PantallaVenta {
    private ControladorVenta controlador;

    public PantallaVenta() {
        controlador = new ControladorVenta();
    }

    public void botonRegistrarVenta() {
        controlador.registrarVenta("Producto A", 2);
    }
}

// Controlador
public class ControladorVenta {
    public void registrarVenta(String producto, int cantidad) {
        Venta venta = new Venta();
        venta.agregarItem(producto, cantidad);
    }
}

// Clase de dominio
public class Venta {
    public void agregarItem(String producto, int cantidad) {
        System.out.println("Agregado " + cantidad + " de " + producto);
    }
}
```

Pros y Contras

Beneficios:

- Separa la lógica de interfaz de usuario de la lógica de negocio.
- Claridad en la estructura: la UI solo muestra datos, el Controlador coordina, las clases de dominio resuelven.
- Facilita el mantenimiento y reutilización de la lógica de negocio.

Contras / Antipatrones:

- Riesgo de **controladores “gordos”**: cuando acumulan demasiada lógica que debería estar en clases de dominio.
- Puede volverse un cuello de botella si no se delega correctamente.

Remedios:

- Aplicar **Experto en Información** para delegar la lógica.
- Mantener el controlador como un **coordinador ligero**, no como un “Dios” que hace todo.

Patrón de Polimorfismo

El problema

El principal problema que el polimorfismo aborda es la rigidez y la complejidad del código que utiliza muchas sentencias **if-then-else** o **switch-case** para manejar diferentes comportamientos basados en el tipo de un objeto. Este enfoque, hace que el código sea difícil de mantener y extender. **Si se agrega un nuevo tipo de objeto, es necesario modificar todas las lógicas condicionales existentes**, lo que lo hace propenso a errores.

- **Analogía en la vida real:** Imaginemos un guardia en la entrada de un edificio de oficinas. Sin polimorfismo, el guardia tiene una lista enorme que dice: *“Si llega un programador, mándalo al piso 3. Si llega un diseñador, al piso 5. Si llega un contador, al piso 7...”*. Cada vez que se contrata un nuevo tipo de trabajador, hay que actualizar **todas las listas de todos los guardias**, y si uno se olvida, la persona termina en el lugar equivocado.

La solución

El patrón de polimorfismo propone una solución elegante: en lugar de preguntar al objeto qué tipo es, simplemente le enviamos el mismo mensaje a todos los objetos y dejamos que cada uno se comporte de la manera que le corresponde. Esto se logra mediante **operaciones polimórficas**.

- **Definición:** El polimorfismo asigna la responsabilidad de un comportamiento que varía según el tipo, directamente a los tipos para los que el comportamiento varía. Su corolario es claro: **No haga comprobaciones acerca del tipo de un objeto y no utilice la lógica condicional para llevar a cabo alternativas diferentes basadas en el tipo.**
- Siguiendo la analogía anterior, con polimorfismo, el guardia no necesita saber nada de quién entra. Solo les dice: *“Ve a tu oficina”*. Cada persona ya sabe a qué piso debe ir. **El guardia no clasifica más: delega la responsabilidad al propio objeto (la persona).**

Tipos o Variantes

El tipo de polimorfismo más relevante para este patrón es el polimorfismo de **subtipos** (también llamado *polimorfismo inclusivo*), que se utiliza en la mayoría de los patrones de diseño orientados a objetos. Su idea central es que una **subclase puede ser usada en lugar de su superclase o de una interfaz**, sin que el código que la usa tenga que preocuparse por el tipo concreto. Esto significa que **podemos “hablarle” a distintos objetos como si fueran del mismo tipo**, y aun así cada uno responderá de manera distinta según lo que es realmente.

Ejemplos

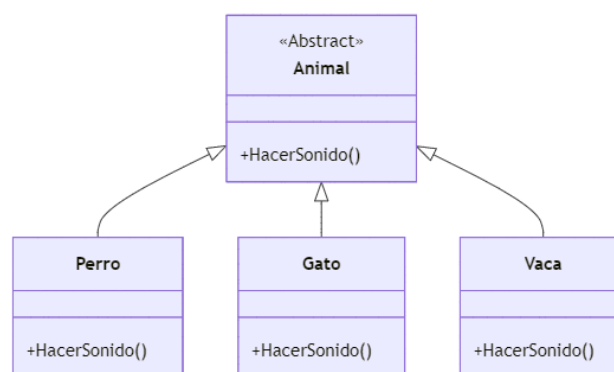
Ejemplo cotidiano

Pensemos en distintos animales: perro, gato y vaca. Si le decimos a todos “*hace tu sonido*”, cada uno responde diferente:

El perro ladra → “Guau”. El gato maúlla → “Miau”. La vaca muge → “Muuu”.

La acción es siempre la misma (*hacer sonido*), pero la respuesta depende del tipo de animal.

Diagrama UML



Animal es la clase abstracta que define el método **HacerSonido()**. **Perro**, **Gato** y **Vaca** heredan de **Animal** e implementan el método de manera distinta. El polimorfismo permite llamar a **HacerSonido()** desde una referencia **Animal**, dejando que cada subclase decida qué sonido producir.

Ejemplo con código

```
Polimorfismo

1 abstract class Animal {
2     abstract void hacerSonido();
3 }
4
5 class Perro extends Animal {
6     void hacerSonido() { System.out.println("Guau"); }
7 }
8
9 class Gato extends Animal {
10    void hacerSonido() { System.out.println("Miau"); }
11 }
12
13 class Vaca extends Animal {
14    void hacerSonido() { System.out.println("Muuu"); }
15 }
16
17 public class Main {
18     public static void main(String[] args) {
19         Animal[] animales = { new Perro(), new Gato(), new Vaca() };
20         for (Animal a : animales) {
21             a.hacerSonido(); // Polimorfismo en acción
22         }
23     }
24 }
25
```

```
Salida

1 Guau
2 Miau
3 Muuu
```

Definimos una clase animal con el método hacerSonido(), y luego las clases hijas (perro, gato, vaca) implementan este método de manera distinta.

Pros y Contras

Beneficios:

- **Flexibilidad y extensibilidad:** Permite agregar nuevos tipos de objetos sin modificar el código que ya los utiliza.
- **Reducción de acoplamiento:** El código cliente depende solo de la interfaz, no de las implementaciones concretas, lo que facilita el mantenimiento.
- **Código limpio:** Elimina la necesidad de complejos if/else, haciendo el código más legible y sencillo.

Desventajas:

- **Inversión de diseño:** A veces, se invierte tiempo en diseñar con polimorfismo para variaciones futuras que pueden nunca llegar a materializarse. Es clave ser realista sobre la necesidad de esta flexibilidad.
- **Antipatrón:** El principal problema es volver al uso de sentencias condicionales. La solución es siempre refactorizar y usar el polimorfismo en su lugar.

Patrón de Fabricación Pura

El problema

El problema en el software: Clases de baja cohesión y alto acoplamiento

El patrón **Fabricación Pura** se aplica para resolver el dilema de a **quién asignar una responsabilidad** cuando ninguna entidad existente del modelo de dominio es el Experto adecuado. Si asignar la responsabilidad a un objeto de dominio comprometería la calidad del diseño al **reducir la Cohesión** o **eleva el Acoplamiento**, el software se volvería frágil y difícil de mantener, lo que demuestra la necesidad de un patrón alternativo para responsabilidades que no encajan lógicamente en el dominio del problema.

Analogía con la vida real: Juegos de mesa

Imaginemos el mundo de los juegos de mesa. Los objetos del dominio del problema son **el Tablero**, los **Jugadores** y **las Fichas**. Cada uno tiene su propia responsabilidad: el Tablero define el espacio de juego y los **Jugadores** toman turnos. El problema surge cuando se necesita una nueva partida. El Tablero no puede crearse a sí mismo, y un Jugador no tiene la responsabilidad de ensamblar un juego completo, con todas sus reglas y piezas. Si se le asignara esta tarea a una de estas entidades, **se sobrecargarían con una responsabilidad ajena a su propósito central**.

La solución

Definición Técnica: Fabricación Pura.

El patrón de **Fabricación Pura** es un principio de asignación de responsabilidades que propone la creación de una clase "inventada" o "artificial" que no representa un concepto en el dominio del problema. Esta clase no tiene un equivalente en el mundo real que se esté modelando, y su único propósito es resolver problemas de diseño de software. Se crea intencionalmente para encapsular una responsabilidad que, de ser asignada a una clase de dominio, resultaría en una baja cohesión y un alto acoplamiento. En esencia, es la solución a un problema de asignación de responsabilidades cuando **el patrón Experto en información** no puede ser aplicado de manera óptima.

Analogía: Crear la entidad CreadorDeJuegos

La solución a la analogía planteada anteriormente es crear un nuevo rol que no existe en el juego, pero que es necesario para que todo funcione: el **CreadorDeJuegos**. Esta entidad no es un Jugador ni una Ficha; es una Fabricación Pura. Su única responsabilidad es saber cómo crear diferentes tipos de juegos de ajedrez o de solitario, para que el Tablero y los Jugadores se enfoquen en sus propias responsabilidades sin acoplarse a los detalles de la creación del juego.

Tipos o Variantes

El patrón Fabricación Pura no tiene tipos o variantes formales, ya que es un principio fundamental de asignación de responsabilidades, no un patrón de diseño con una implementación única. En su lugar, es un concepto que se manifiesta en muchos otros patrones de diseño, como los del catálogo de la "**Gang of Four**" (GoF).

La Fabricación Pura es el principio subyacente que explica por qué la creación de estas clases artificiales es necesaria para lograr un diseño de software de alta calidad.

Ejemplos de patrones que aplican el concepto de Fabricación Pura incluyen:

Patrones de Creación (como Factory Method, Abstract Factory, y Builder), que usan clases "fabricadas" para gestionar la creación de objetos.

Patrones de Comportamiento (como Strategy), que usan clases inventadas para encapsular algoritmos.

Patrones de Estructura (como Adapter y Decorator), que usan clases artificiales para resolver problemas de compatibilidad o añadir funcionalidades

Ejemplos

Ejemplo cotidiano: Persistencia de datos.

Para ilustrar el patrón de Fabricación Pura, se puede usar un ejemplo de persistencia de datos. El problema es que una clase de dominio como **Producto** no debería ser responsable de guardar su propia información en una base de datos. Se introduce la clase **ProductoRepository**, una **Fabricación Pura** cuya única responsabilidad es gestionar la persistencia de los objetos Producto. De esta forma, la clase Producto delega las tareas de base de datos al repositorio, manteniendo su enfoque y reduciendo el acoplamiento.

```
// Clase de dominio, enfocada en la lógica de negocio del producto
public class Producto {
    // Atributos que representan las características de un producto
    private String nombre;
    private double precio;

    // Constructor
    public Producto(String nombre, double precio) {
        this.nombre = nombre;
        this.precio = precio;
    }

    // Métodos (getters y setters) para acceder a los atributos
    public String getNombre() {
        return nombre;
    }

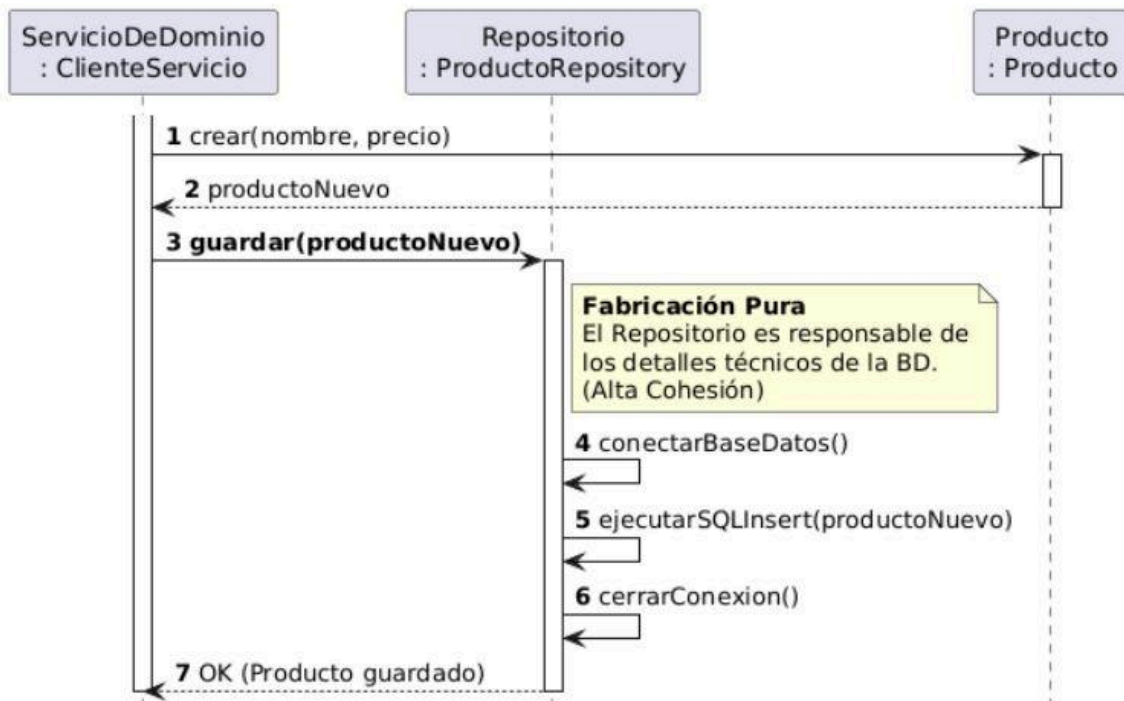
    public double getPrecio() {
        return precio;
    }
}

// El objeto Producto no sabe cómo guardarse a sí mismo

// Fabricación Pura: Clase para gestionar la persistencia de los productos
// Esta clase existe solo para separar la lógica de base de datos
public class ProductoRepository {

    // Método para guardar un producto en la "base de datos"
    // El repositorio se encarga de los detalles técnicos de almacenamiento
    public void guardar(Producto producto) {
        System.out.println("Guardando el producto: " + producto.getNombre() + " en la base de datos.");
        // Lógica de base de datos iría aquí (SQL, JPA, etc.) [26, 27]
    }
}
```

Patrón Fabricación Pura: Persistencia



Pros y Contras

Beneficios

- **Alta Cohesión:** Al crear una clase con una única y bien definida responsabilidad, el patrón asegura que cada clase del sistema se mantenga enfocada en un solo propósito. Esto hace que el código sea más fácil de entender y mantener.²
- **Bajo Acoplamiento:** Es la principal ventaja del patrón. Al centralizar una responsabilidad en una clase fabricada, los objetos del dominio se desacoplan de los detalles de implementación específicos. Un cambio en una clase fabricada (por ejemplo, el tipo de base de datos o el formato de log) no afectará a las clases de dominio, siempre y cuando se mantenga la interfaz de comunicación.²
- **Reutilización:** Las clases de Fabricación Pura son altamente reutilizables. Ya que encapsulan una funcionalidad cohesiva y genérica que puede ser adaptada a diferentes contextos sin grandes modificaciones.

Desventajas

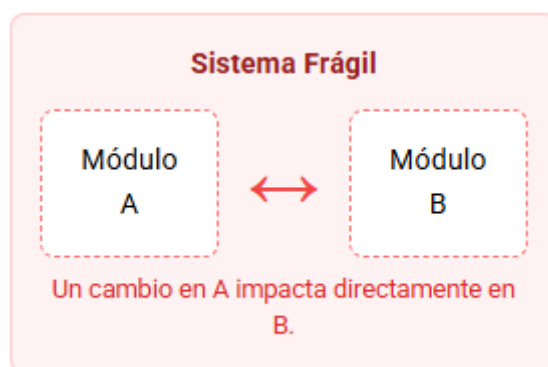
Riesgo de Diseño Procedural/Funcional: Si se abusa del patrón, se puede caer en el error de "convertir funciones en objetos" de manera superficial. El resultado son clases sin estado, que solo agrupan métodos y manipulan objetos de dominio que no tienen comportamiento propio. Esto va en contra de la esencia de la programación orientada a objetos, ya que el sistema se vuelve una colección de procedimientos en lugar de objetos que interactúan.

Patrón de Indirección

El problema

En el Software: Alto Acoplamiento

Cuando dos o más componentes de un sistema están fuertemente conectados, cualquier cambio en uno de ellos obliga a modificar a los otros. Esto genera un software frágil, difícil de mantener, probar y reutilizar. A este problema se le conoce como "alto acoplamiento".



Analogía: Comprar en el Extranjero

Imagina que quieres comprar un producto de una tienda en Japón desde Argentina. Si tuvieras que pagarles directamente, tendrías que:

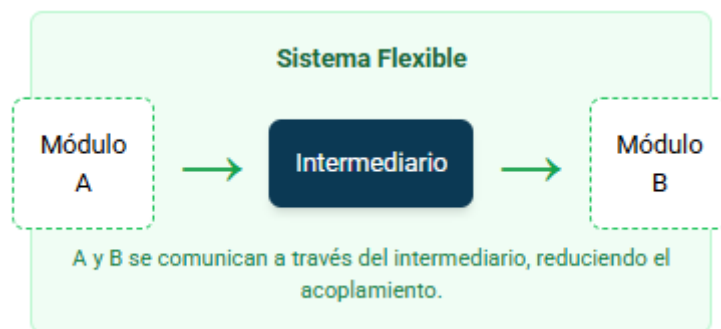
- Conseguir Yenes Japoneses.
- Entender y cumplir las leyes bancarias de Japón.
- Realizar una transferencia internacional compleja.
- Coordinar la operación en diferentes zonas horarias.

Estás "fuertemente acoplado" a los procesos y requisitos específicos de esa tienda.

La solución

Definición Técnica: El Intermediario

El patrón Indirección resuelve el problema del alto acoplamiento asignando la responsabilidad de la comunicación entre dos componentes a un objeto intermediario. Este intermediario desacopla a los componentes originales, que ya no se conocen directamente entre sí.



Analogía: Usando una Tarjeta de Crédito

Para evitar la complejidad, usas tu tarjeta de crédito (Visa, Mastercard). La tarjeta actúa como un intermediario.

- Tú pagas a la tarjeta en Pesos Argentinos.
- La tarjeta se encarga de convertir la moneda y pagarle a la tienda japonesa.
- Tú no necesitas saber nada sobre bancos japoneses; solo cómo usar tu tarjeta.

La tarjeta de crédito (el objeto de indirección) ha desacoplado al comprador del vendedor, simplificando la interacción.

Tipos o Variantes

El patrón **Indirección** es un principio fundamental que se manifiesta a través de otros patrones de diseño más específicos. No tiene "variantes" formales, sino implementaciones concretas. Aquí se detallan dos de las más comunes:

Fachada

El patrón **Fachada** proporciona una interfaz unificada y simplificada a un conjunto de interfaces en un subsistema. Define un punto de entrada de alto nivel que hace que el subsistema sea más fácil de usar.

Adaptador (Adapter)

El patrón **Adaptador** (Adapter) permite que interfaces incompatibles trabajen juntas. Actúa como un traductor entre dos clases, convirtiendo la interfaz de una clase en otra que el cliente espera.

Ejemplos

Ejemplo Cotidiano: El Enchufe Eléctrico

No conectes tu notebook directamente a los cables de alta tensión de la red eléctrica. El enchufe, la toma de corriente y el transformador de tu cargador actúan como capas de indirección.

Estos intermediarios adaptan el voltaje, proveen una interfaz estándar y se desacoplan de la complejidad y el peligro de la red eléctrica.

Ejemplo de Código: Terminal de Venta

Un objeto ``Venta`` necesita el precio de un producto. En lugar de que ``Venta`` consulte directamente la base de datos (alto acoplamiento), le pide la información a un objeto ``CatalogoProductos``. Este catálogo es el intermediario.

```

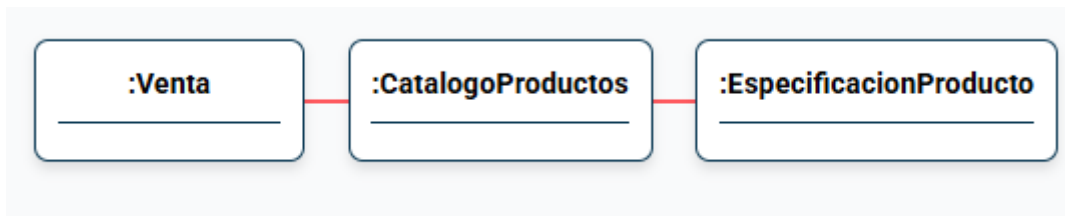
public class Venta {
    private CatalogoProductos catalogo;

    public Venta(CatalogoProductos catalogo) {
        this.catalogo = catalogo;
    }

    public void agregarItem(int idProducto, int cantidad) {
        // Venta no sabe cómo se obtiene el producto.
        // Se lo pide al intermediario (catalogo).
        EspecificacionProducto producto = catalogo.getEspecificacion(idProducto);
        // ... usar producto para calcular el total
    }
}

```

Diagrama UML de la Solución



La clase `Venta` solo se comunica con `CatalogoProductos`, el cual actúa como intermediario para obtener los datos del producto. `Venta` y `EspecificacionProducto` están desacoplados.

Pros y Contras

Beneficios

- **Menor Acoplamiento:** Los componentes del sistema se vuelven más independientes entre sí. Al no tener conocimiento directo unos de otros, los cambios en un componente no afectan necesariamente a los demás, siempre y cuando el intermediario mantenga el contrato de comunicación.
- **Mayor Reutilización:** Al estar desacoplados, los componentes individuales son como piezas. Se pueden tomar y usar en diferentes partes del sistema o incluso en otros proyectos sin necesidad de grandes modificaciones. El intermediario se encarga de adaptarlos al contexto específico.
- **Facilita las Pruebas Unitarias:** Permite aislar componentes para probarlos de forma independiente.

Desventajas (Contras)

- **Mayor complejidad:** La introducción de intermediarios inevitablemente agrega más "piezas móviles" al sistema, como clases o módulos adicionales. Para alguien que no conoce el diseño, el flujo de comunicación puede ser más difícil de seguir, ya que no es una línea recta, sino que pasa por uno o más intermediarios.
- **Sobrecarga de rendimiento (Performance Overhead):** Cada capa de indirección añade un pequeño paso extra en la ejecución del código. En la mayoría de las aplicaciones, esta sobrecarga es mínima y completamente aceptable. Sin embargo, en sistemas de muy alto rendimiento, como videojuegos o aplicaciones de trading de alta frecuencia, cada nanosegundo cuenta, y la indirección podría convertirse en un cuello de botella.
- **Antipatrón - Indirección Excesiva:** Abusar de este patrón es contraproducente. Añadir capas y capas de intermediarios sin una razón clara complica el código innecesariamente, dificultando su depuración y entendimiento. Es como pedirle a tu asistente que le pida a su asistente que envíe un correo: se añade burocracia sin aportar valor real.

Patrón de Variaciones Protegidas

El problema

En un sistema de software siempre existen puntos de inestabilidad: requisitos que cambian, APIs externas, reglas de negocio que se actualizan, nuevos cálculos o integraciones. La pregunta clave es: **¿Cómo diseñar objetos y subsistemas para que las variaciones o inestabilidades no afecten negativamente al resto del sistema?** Un mal diseño provoca que cada cambio en una parte (ej: API externa) impacte en múltiples clases internas, generando alto acoplamiento y alto costo de mantenimiento.

Analogía en la vida real:

Imaginá que enchufás tus dispositivos en casa. Si cada aparato requiriera un enchufe distinto, tendrías que rehacer toda la instalación eléctrica cada vez que cambias de electrodoméstico. En cambio, usamos **enchufes estándar**: lo que cambia es el adaptador del aparato, no la instalación de tu casa. **Lo mismo busca este patrón: aislar el cambio detrás de un punto estable.**

La solución

Definición técnica:

El patrón Variaciones Protegidas (VP) propone:

- Identificar los puntos de variación, es decir, lugares donde es más probable que ocurran cambios.
- Encapsular esas variaciones detrás de una interfaz estable.
- Los clientes dependen de la interfaz estable, no de las implementaciones variables.

Esto protege al sistema del impacto de cambios externos, y al mismo tiempo facilita la extensibilidad con bajo acoplamiento.

Analogía (siguiendo la anterior):

La instalación eléctrica (interfaz estable) no cambia. Solo cambian los adaptadores de cada dispositivo (implementaciones variables).

Tipos o Variantes

El patrón Variaciones Protegidas no tiene una clasificación formal, sino que se pueden distinguir en distintos enfoques:

1. **Protección mediante Interfaces (Polimorfismo):**
 - Se define una interfaz y varias implementaciones posibles.
 - Útil cuando se esperan múltiples variaciones de comportamiento.
2. **Protección mediante Indirección (Intermediarios):**
 - Se introduce un objeto intermediario (adaptador, proxy, fachada).
 - Útil cuando se quiere desacoplar dependencias externas o APIs inestables.
3. **Protección mediante Encapsulación de Datos:**
 - Se ocultan estructuras internas, exponiendo solo lo necesario.
 - Útil cuando se busca proteger a los clientes de cambios en la representación interna.

Ejemplos

Ejemplo cotidiano

Una caja fuerte con combinación:

- El mecanismo interno puede variar (electrónico, mecánico, biométrico).
- Lo que no cambia es la interfaz estable: el usuario siempre abre la caja introduciendo una clave o medio de acceso.

Ejemplo en software (nuevo: sistema de pagos)

En un sistema de e-commerce, se deben soportar distintos métodos de pago: tarjeta de crédito, PayPal, transferencia bancaria, criptomonedas, etc.

- Problema: Si el sistema se conecta directamente a cada API, cualquier cambio rompe el código.
- Solución con VP: Crear una interfaz estable `IMetodoPago`. Cada forma de pago se implementa de manera independiente. El sistema de compras solo conoce `IMetodoPago`, no a las clases concretas.

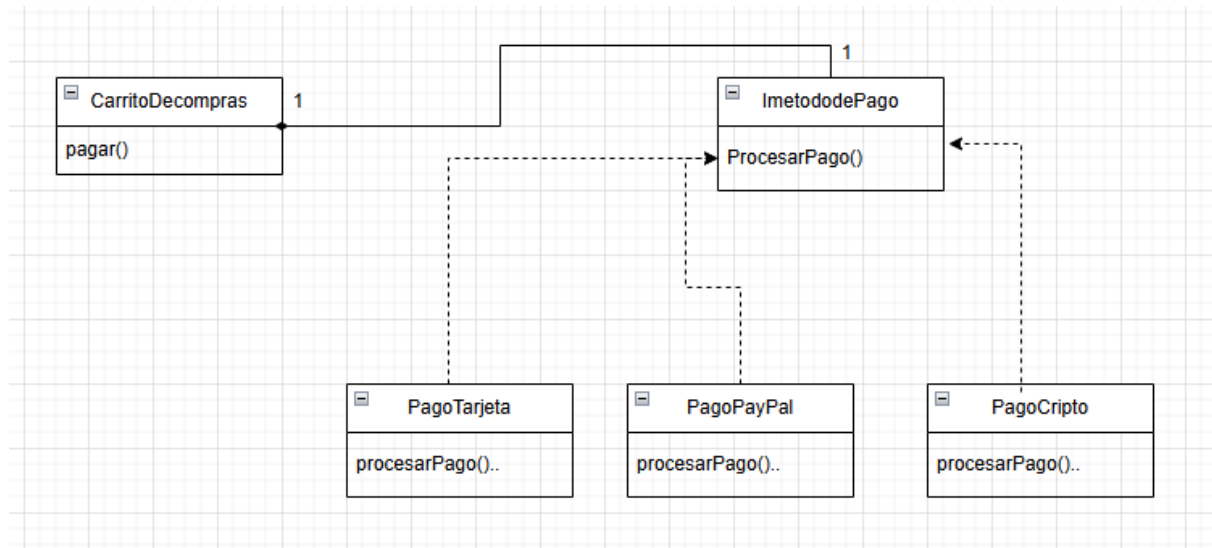
```
// Interfaz estable
public interface IMetodoPago {
    boolean procesarPago(double monto);
}

// Implementación concreta: Tarjeta
public class PagoTarjeta implements IMetodoPago {
    @Override
    public boolean procesarPago(double monto) {
        System.out.println("Procesando pago con tarjeta por $" + monto);
        return true;
    }
}

// Implementación concreta: PayPal
public class PagoPayPal implements IMetodoPago {
    @Override
    public boolean procesarPago(double monto) {
        System.out.println("Procesando pago con PayPal por $" + monto);
        return true;
    }
}

// Implementación concreta: Criptomonedas
public class PagoCripto implements IMetodoPago {
    @Override
    public boolean procesarPago(double monto) {
        System.out.println("Procesando pago en criptomonedas por $" + monto);
        return true;
    }
}
```

Diagrama UMI Solución:



Pros y Contras

Beneficios:

- Se añaden nuevas variaciones sin modificar el código cliente.
- Reduce el acoplamiento.
- Disminuye impacto y costo de cambios.
- Aumenta la flexibilidad y la reutilización.

Desventajas:

- Puede generar sobre-ingeniería (añadir capas innecesarias).
- Riesgo de proteger variaciones irreales (diseñar para cambios que nunca ocurrirán).
- Mayor complejidad inicial en sistemas pequeños.

Conclusión

El patrón Variaciones Protegidas nos recuerda que el cambio es inevitable en el software: proveedores externos modifican sus APIs, las reglas de negocio evolucionan y las tecnologías se reemplazan. La clave está en aislar esos puntos de variación mediante interfaces estables, de modo que el sistema no se vea afectado directamente.

Aplicarlo bien significa encontrar un equilibrio: proteger lo que realmente es inestable sin caer en la sobre-ingeniería. Cuando se utiliza con criterio, este patrón aumenta la flexibilidad, el bajo acoplamiento y la mantenibilidad del diseño, permitiendo que el sistema crezca y se adapte con menor costo y menor riesgo.

Fuentes

- Larman, C. *Applying UML and Patterns*. 3rd Edition. Prentice Hall.
- Notas de cátedra de Diseño de Sistemas (Unidad 5 – GRASP).
- Craig Larman, *GRASP: Patterns for Assigning Responsibilities*.